

The Coin

Uma moeda digital de prova de trabalho, leve e democrática,
com contratos inteligentes, privacidade opcional e governança on-chain

Whitepaper — versão 0.2

13 de setembro de 2026 · the-coin.cloud

substitui a versão 0.1 (setembro de 2026), que permanece publicada

Resumo. The Coin (TCN) é uma rede monetária aberta e ponto a ponto, projetada para que máquinas modestas — um VPS de 1 vCPU e 1 GB de RAM — possam validar a cadeia completa e competir de forma justa pelas recompensas de mineração. A prova de trabalho **CoinHash** usa Argon2id com 16 MiB de memória, e a dificuldade é recalculada a cada bloco (LWMA-1). A oferta é limitada a 50 milhões de TCN, sem pré-mineração, com 93,75% emitidos em $\approx 4,75$ anos. A versão 0.2 introduz um **modelo de taxas** com base fixa, custo por byte e por combustível e um multiplicador de congestionamento cuja sobretaxa é **queimada**; **níveis de prioridade**; **confirmação rápida e segura** (substituição por taxa opcional, alertas de gasto duplo, blocos compactos, estimador de confirmações e recompensas com liberação escalonada); contratos inteligentes na linguagem **TCCL**, com execução determinística medida por combustível e depósito de armazenamento reembolsável; **privacidade opcional** por assinaturas em anel dentro de contratos; armazenamento mais compacto; e instalação de validador em uma linha. Apresenta ainda o plano aprovado de evolução do consenso de PoW para um modelo híbrido e, depois, para prova de participação — um plano, não uma funcionalidade desta versão.

Sumário

1	Introdução	1
2	O que mudou na versão 0.2	1
3	Visão geral do sistema	3
4	Modelo de dados	3
4.1	Unidades	3
4.2	Codificação canônica e hashes	3
4.3	Contas e endereços	4
4.4	Transações	4
4.5	Blocos	4
5	Consenso por prova de trabalho	5
5.1	CoinHash	5
5.2	Dificuldade justa (LWMA-1)	5
5.3	Regras de tempo	6
5.4	Escolha da cadeia e finalidade	6
6	Política monetária	6
6.1	Emissão	6
6.2	Por que um ciclo de cinco anos	7
6.3	Liberação da recompensa, queima e oferta circulante	7
7	Taxas	8
7.1	O modelo	8
7.2	Multiplicador de congestionamento	8
7.3	Por que este modelo	9
7.4	Exemplos com os parâmetros padrão da mainnet	9
7.5	Prioridade: pagar mais para confirmar antes	10
8	Confirmação rápida e segura	10
8.1	Substituição por taxa opcional	11
8.2	Alertas de gasto duplo	11
8.3	Blocos compactos	11
8.4	Quantas confirmações esperar	11
8.5	Reorganização máxima e liberação escalonada da recompensa	11
9	Estado global e armazenamento compacto	12
9.1	Registros de estado	12
9.2	Compromisso homomórfico de estado (LtHash)	12
9.3	Banco de dados (esquema 2)	12
10	Transações e pagamentos	13
11	Contratos de pagamento nativos	13
12	Contratos inteligentes TCCL	14
12.1	Princípios de projeto	14
12.2	Exemplo	15
12.3	Recibos e eventos	15
12.4	Custos e limites	15
12.5	Depósito de armazenamento reembolsável	16
12.6	Liberdade total para desenvolvedores	16
13	Privacidade opcional por contratos	17
13.1	Assinaturas em anel (bLSAG)	17
13.2	Pool de valor fixo	17
13.3	Limites do anonimato	18
14	Governança bicameral	18
14.1	Parâmetros governáveis	18
14.2	Sinalização de mineradores	19

15	Rede ponto a ponto	19
16	Nó, mineração e instalação	20
16.1	Modelo de execução	20
16.2	Virar validador com uma linha	20
16.3	O comando <code>thecoin</code>	21
16.4	Requisitos	21
17	Carteira e padrões para desenvolvedores	21
18	API e integração com o site	22
19	Segurança	22
20	Desempenho e escalabilidade	23
20.1	Capacidade de processamento	23
20.2	Armazenamento: o que ocupa espaço	24
20.3	Projeções de disco	24
20.4	Mempool persistente e sincronização	25
21	Roadmap de consenso: PoW → Híbrido → PoS	25
21.1	Por que esta ordem	26
21.2	Fase 1 — PoW (atual)	26
21.3	Fase 2 — Híbrido: PoW + finalidade PoS	26
21.4	Fase 3 — PoS com finalidade rápida	27
21.5	O que não muda em nenhuma fase	27
21.6	Riscos e tratamento	27
22	Lançamento da rede e roteiro de software	28
22.1	Plano de lançamento	28
22.2	Roteiro	28
23	Conclusão	28
	Apêndice A — Parâmetros das redes	29
	Apêndice B — Referências	29

1 Introdução

O Bitcoin demonstrou que é possível manter um livro-razão público, resistente à censura e à falsificação, sem autoridade central. Quinze anos depois, porém, a mineração tornou-se uma indústria de ASICs especializados e energia barata, e rodar um nó completo exige recursos que afastam o usuário comum. Redes com contratos inteligentes, por sua vez, frequentemente trocaram a descentralização por desempenho, exigindo servidores potentes para validar a cadeia.

The Coin parte de uma pergunta simples: **e se a unidade mínima de participação na rede fosse um VPS barato?** Milhões de servidores virtuais de baixo custo existem no mundo inteiro. Se cada um deles puder validar blocos, retransmitir transações e minerar com chances proporcionais ao seu trabalho, a rede se torna mais distribuída geograficamente, mais difícil de capturar e mais democrática.

Os objetivos do projeto são:

- **Leveza:** validar a cadeia completa com 1 vCPU, 1 GB de RAM e poucos GB de disco.
- **Segurança:** regras de consenso simples, determinísticas e auditáveis; nenhuma transação confirmada pode ser alterada sem refazer o trabalho acumulado.
- **Mineração justa:** prova de trabalho **memory-hard** amigável a CPUs comuns e dificuldade que reage rápido à entrada e saída de máquinas.
- **Política monetária previsível:** 50 milhões de TCN, sem pré-mineração, emissão por halvings.
- **Taxas baratas e resistentes a abuso:** custo proporcional aos recursos usados, que sobe sozinho quando a rede é inundada e volta a cair quando a demanda passa.
- **Utilidade:** pagamentos, contratos de pagamento, contratos inteligentes gerais e privacidade opcional — sem abrir mão de validação barata.
- **Evolução democrática:** mudanças de parâmetros aprovadas on-chain por detentores e mineradores.
- **Abertura:** especificação, API e padrões de carteira documentados para que qualquer pessoa crie carteiras, exploradores, contratos e implementações alternativas.

Status. Este documento descreve a implementação de referência **v0.2.0** em Rust (repositório [LucasBolla94/thecoin](https://github.com/LucasBolla94/thecoin)). Todos os números de protocolo citados foram conferidos no código-fonte (`crates/core/src/params.rs`, `execution.rs`, `programs.rs` e demais). Onde houver divergência, o código e a especificação `docs/PROTOCOL.md` são normativos. A Seção 21 descreve um **plano** de consenso que não está implementado nesta versão.

2 O que mudou na versão 0.2

A tabela a seguir resume as mudanças em relação ao whitepaper v0.1. Cada item é detalhado nas seções indicadas; o restante do documento mantém o texto da v0.1 onde ele continua válido.

Área	v0.1	v0.2
Taxas (Seção 7)	<code>min_fee_per_byte</code> × ; 100% tamanho ao minerador.	<code>(base + por kB + por combustível) × congestionamento</code> ; a sobretaxa de congestionamento é queimada; o excedente (gorjeta) vai ao minerador.
Prioridade (Seção 7.5)	Sugestão pela mediana do mempool.	Níveis <code>low</code> , <code>normal</code> , <code>high</code> , <code>urgent</code> ; ordenação por taxa por unidade de peso.
Substituição por taxa (Seção 8)	Qualquer transação pendente, com +25%.	Somente com a flag <code>replaceable</code> (opt-in) e +25%; tentativa sem a flag vira

Área	v0.1	v0.2
		alerta de gasto duplo propagado na rede.
Formato de transação	Sem campo de opções.	Campo <code>flags</code> (u8); ações <code>Deploy</code> e <code>Invoke</code> para TCCL. Transferência simples: 158 → 159 bytes.
Recompensa de bloco	Maturação única de 100 blocos.	25% liberados após 100 blocos, 75% após 1 000 blocos (além do limite de reorganização de 720).
Dificuldade (Seção 5.2)	LWMA-1 só após 61 blocos.	Aquecimento a partir de 6 intervalos; variação máxima de 2× por bloco.
Contratos (Seção 12)	Cinco modelos nativos de pagamento.	+ linguagem TCCL: compilador no consenso, VM determinística com combustível, eventos e recibos; depósito de armazenamento reembolsável (também nos modelos nativos; <code>MultisigClose</code>).
Privacidade (Seção 13)	—	Assinaturas em anel bLSAG (Ristretto255) acessíveis a contratos; pool de valor fixo; comandos <code>privacy</code> na carteira.
Rede P2P	Anúncio <code>Inv</code> e download do bloco completo.	Blocos compactos com IDs curtos de 6 bytes; mensagens <code>DoubleSpend</code> .
Segurança de pagamentos	Recomendação fixa de 10/60 confirmações.	Estimador por valor (<code>/api/v1/security</code> , <code>thecoin-wallet confirmations</code>); alertas (<code>/api/v1/alerts</code>).
Armazenamento (Seção 9)	Esquema 1; índice de endereços guardava o txid.	Esquema 2; tabela de recibos; índice de endereços sem valor; poda remove índice de transações e recibos; mempool persistente; <code>thecoin compact</code> .
Governança (Seção 14)	8 parâmetros.	12 parâmetros (taxas, combustível, depósito); sinalização por <code>thecoin --signal</code> e <code>thecoin signal</code> .
Instalação (Seção 16)	Instalador interativo.	Uma linha, não interativa (<code>--yes</code>); comando auxiliar <code>thecoin</code> ; ferramenta <code>tccl</code> instalada.
API	Consultas e envio.	+ <code>/tx/simulate</code> , <code>/program/{addr}</code> , <code>/program/{addr}/view</code> , <code>/security</code> , <code>/alerts</code> ; <code>/fees</code> com níveis de prioridade.
Roteiro (Seção 21)	Noise, snapshot, VM WebAssembly.	Plano aprovado PoW → Híbrido → PoS; TCCL substitui a VM WebAssembly prevista.

Tabela 1: Resumo das mudanças v0.1 → v0.2.

3 Visão geral do sistema

A rede é formada por nós idênticos (`thecoind`). Cada nó mantém a cadeia de blocos, valida cada regra de consenso de forma independente, retransmite blocos e transações pela rede ponto a ponto e, opcionalmente, minera. Neste documento, **validador** é um nó completo que valida tudo e minera — é o que o instalador de uma linha configura. Carteiras (`thecoin-wallet` ou de terceiros) guardam as chaves privadas localmente e só enviam transações já assinadas a um nó.

Componente	Responsabilidade
<code>thecoin-core</code>	Regras de consenso puras: tipos, codificação canônica, criptografia, CoinHash, LWMA, emissão, taxas e congestionamento, contratos nativos, execução de contratos TCCL, governança e função de transição de estado. Sem I/O.
<code>tccl</code>	The Coin Cloud Language: analisador, verificador de tipos e compilador, máquina virtual determinística com combustível, assinaturas em anel, simulador e a ferramenta de linha de comando <code>tccl</code> .
<code>thecoin-storage</code>	Banco de dados embarcado redb (ACID), blocos, undo e recibos comprimidos com <code>zstd</code> , índices de transações e endereços.
<code>thecoin-node</code>	Gerenciador da cadeia (validação, escolha do fork, reorganizações, poda), mempool persistente com prioridade e alertas de gasto duplo, rede P2P com blocos compactos, minerador de CPU e API REST.
<code>thecoin-wallet</code>	Chaves HD (BIP-39/SLIP-0010), arquivo de carteira criptografado, construção e assinatura de transações, prioridade de taxa, contratos, privacidade, governança, URIs de pagamento, CLI.
Instalador e <code>thecoin</code>	Script de uma linha que instala binários verificados, cria carteira, configura serviço <code>systemd</code> endurecido e inicia a mineração; comando auxiliar para operar o nó.
Site / explorador	<code>the-coin.cloud</code> : estatísticas ao vivo, explorador de blocos, painel de governança, guia do validador e downloads, alimentado pela API dos nós.

Tabela 2: Componentes da implementação de referência.

O fluxo básico é o mesmo do Bitcoin: usuários assinam transações; nós as validam e as guardam no **mempool**; mineradores agrupam as transações que pagam mais por recurso usado e procuram um **nonce** que satisfaça a prova de trabalho; o bloco encontrado é propagado; cada nó o valida integralmente e, se ele estender a cadeia com mais trabalho acumulado, o adota.

4 Modelo de dados

4.1 Unidades

A unidade de conta é o **TCN**. A menor fração é o **mote**: 1 TCN = 100 000 000 motes. Todos os valores são inteiros de 64 bits sem sinal; ponto flutuante nunca é usado no consenso. A oferta máxima (5×10^{15} motes) cabe com folga em 64 bits e também é um inteiro seguro em JavaScript, o que simplifica carteiras web.

4.2 Codificação canônica e hashes

Todas as estruturas de consenso são serializadas com **Borsh**: inteiros little-endian de tamanho fixo, vetores e strings prefixados por comprimento `u32`, enums com tag `u8`. Cada objeto tem exatamente uma codificação válida, o que impede ambiguidades na assinatura e no hash.

Todo hash do protocolo usa BLAKE3 com separação de domínio:

$$H_{\text{tag}(x)} = \text{BLAKE3}(\text{"TheCoin:"} \parallel \text{tag} \parallel 0x00 \parallel x)$$

Tags distintas (`txid`, `block`, `tx-sign`, `merkle-leaf`, `address`, ...) garantem que um valor calculado para um propósito jamais colida com outro.

4.3 Contas e endereços

The Coin usa o **modelo de contas**: cada endereço possui saldo e **nonce** (número de transações já enviadas). Comparado ao modelo UTXO, o estado é menor, a validação de contratos é direta e carteiras leves precisam consultar um único registro.

O endereço é formado pelos 20 primeiros bytes de H_{address} (chave pública Ed25519) e representado em **bech32m** (BIP-350), com prefixo por rede: `tcl...` (mainnet), `tctl...` (testnet) e `tcr1...` (regtest). O checksum cobre o prefixo, então um endereço de outra rede ou com erro de digitação é rejeitado. Contratos TCCL também têm endereço (derivado de criador e nonce) e saldo próprio, como qualquer conta.

4.4 Transações

Campo	Tipo	Descrição
<code>version</code>	u8	Versão do formato (1).
<code>chain_id</code>	u32	Identificador da rede — impede reutilizar uma transação em outra rede.
<code>flags</code>	u8	Opções. Bit 0 = <code>replaceable</code> (aceita substituição por taxa). Bits desconhecidos invalidam a transação.
<code>nonce</code>	u64	Deve ser igual ao nonce atual da conta; impede replay.
<code>fee</code>	u64	Taxa em moedas; deve cobrir a taxa mínima (Seção 7). O excedente é gorjeta de prioridade.
<code>expiry_height</code>	u64	Última altura em que pode ser incluída (0 = sem validade).
<code>action</code>	enum	Transferência, lote, criar/chamar contrato nativo, propor, votar, publicar (<code>Deploy</code>) ou chamar (<code>Invoke</code>) contrato TCCL.
<code>public_key</code>	32 bytes	Chave pública Ed25519 do remetente.
<code>signature</code>	64 bytes	Assinatura Ed25519 sobre $H_{\text{tx-sign}}$ (body).

Tabela 3: Estrutura de uma transação.

O identificador é $\text{txid} = H_{\text{txid}}$ (transação serializada). A verificação Ed25519 é **estrita** (RFC 8032, rejeitando chaves de ordem pequena e assinaturas não canônicas), eliminando a maleabilidade: o txid só muda se o dono da chave assinar outra transação.

Uma propriedade útil para carteiras: como `fee` tem largura fixa, o tamanho da transação não depende do valor da taxa. Basta assinar uma vez com taxa zero para descobrir o tamanho exato e então assinar com a taxa final. Uma transferência simples sem memo ocupa **159 bytes**.

As transações nativas (transferências, lotes, contratos de pagamento e governança) são **atômicas**: ou são totalmente válidas e aplicadas, ou não podem entrar em um bloco — não existe “falhou mas cobrou taxa”. A exceção deliberada são as transações de contratos TCCL (`Deploy`, `Invoke`): se o código do contrato falhar, a taxa é cobrada e todos os demais efeitos são revertidos; do contrário, qualquer um poderia fazer a rede executar código que falha sem pagar nada (Seção 12).

4.5 Blocos

Campo	Tamanho	Descrição
<code>version</code>	4	Versão do cabeçalho (1).
<code>height</code>	8	Altura do bloco (gênese = 0).
<code>prev_hash</code>	32	Hash do cabeçalho anterior.
<code>tx_root</code>	32	Raiz Merkle dos txids.

Campo	Tamanho	Descrição
state_root	32	Compromisso do estado global após aplicar o bloco (LtHash).
timestamp	8	Horário Unix em segundos.
target	32	Alvo da prova de trabalho (inteiro de 256 bits big-endian).
nonce	8	Campo livre para a busca da prova de trabalho.
miner	20	Endereço que recebe recompensa e taxas.
signal	4	Bits de sinalização de governança dos mineradores.

Tabela 4: Cabeçalho de bloco: tamanho fixo de 180 bytes.

O identificador do bloco é H_{block} (cabeçalho), barato de calcular. A prova de trabalho é um hash separado e caro (CoinHash). Não existe transação **coinbase**: a recompensa é definida pelo campo `miner` e registrada no estado. Um bloco tem dois limites, ambos parâmetros de governança: tamanho serializado (`max_block_bytes`, 1 MB) e soma do combustível reservado pelas transações de contrato (`max_block_fuel`, 50 milhões).

A árvore Merkle usa folhas e nós com tags distintas e promove o último elemento de níveis ímpares sem duplicá-lo, eliminando a ambiguidade conhecida do Bitcoin (CVE-2012-2459).

5 Consenso por prova de trabalho

5.1 CoinHash

$\text{CoinHash}(\text{cabeçalho}) = \text{Argon2id}(\text{senha} = \text{cabeçalho}, \text{sal} = \text{"TheCoin/PoW/v1.."}, m = 16 \text{ MiB}, t = 1, p = 1)$

O resultado de 32 bytes, lido como inteiro big-endian, deve ser menor ou igual ao `target` do cabeçalho. Argon2id (RFC 9106 [2]) é o vencedor da Password Hashing Competition e é **memory-hard**: cada tentativa exige preencher 16 MiB de memória em ordem dependente dos dados. As consequências são:

- **CPUs comuns são competitivas.** O gargalo é a largura de banda de memória, não o número de transistores de lógica; GPUs e ASICs têm vantagem muito menor do que em SHA-256.
- **Verificação barata para nós modestos.** Um hash custa milissegundos, e cada nó verifica apenas um hash por bloco.
- **Memória previsível.** Cada thread de mineração usa 16 MiB; um VPS de 1 GB minera sem pressão.

Memória Argon2id	Tempo por hash	Hashes/s por núcleo
4 MiB	2,41 ms	415
8 MiB	5,22 ms	192
16 MiB (escolhido)	12,04 ms	83
32 MiB	25,16 ms	40

Tabela 5: Medições em um VPS OVH de 2 vCPU (Haswell). 16 MiB equilibra resistência a hardware especializado e custo de verificação.

Com 83 hashes/s, verificar a prova de trabalho de um ano inteiro de blocos (525 960 blocos) custa cerca de 1,75 hora de CPU num único núcleo; o nó verifica blocos recebidos em paralelo em todos os núcleos durante a sincronização, mantendo a ordem de aplicação.

5.2 Dificuldade justa (LWMA-1)

Redes jovens de máquinas pequenas sofrem grandes variações de **hashrate** à medida que mineradores entram e saem. The Coin recalcula o alvo a cada bloco com a média móvel linearmente ponderada LWMA-1 (Zawy [7]), com janela $N = 60$ blocos e tempo-alvo $T = 60$ s. Para o bloco de altura h , usa-se $n = \min(N, h - 1)$ intervalos:

$$t_j = \min(6T, \max(s_j, s_{j-1} + 1) - s_{j-1})$$

$$\text{alvo}^* = \frac{\sum_{j=1}^n \text{alvo}_j}{n} \cdot \frac{\sum_{j=1}^n j \cdot t_j}{k}, \quad k = n(n+1) \frac{T}{2}$$

$$\text{alvo}_h = \min\left(\text{pow_limit}, \max\left(\frac{\text{alvo}_{h-1}}{2}, \min(2 \cdot \text{alvo}_{h-1}, \text{alvo}^*)\right)\right)$$

Três regras tornam o ajuste **justo**:

1. **Reação rápida.** Blocos recentes pesam mais: se uma máquina grande entra ou sai, a dificuldade acompanha em minutos. Ninguém ganha uma rajada de blocos baratos ao ligar muito **hashrate** de uma vez, e os pequenos mineradores não ficam presos a uma dificuldade alta depois que ele sai.
2. **Aquecimento.** Na v0.1 os primeiros 61 blocos usavam o alvo da gênese. Na v0.2 o ajuste começa assim que existem **6 intervalos** (`LWMA_MIN_WINDOW`), usando todos os intervalos disponíveis até completar a janela — uma rede recém-lançada reage após poucos blocos.
3. **Limite de 2× por bloco.** O novo alvo nunca é mais que 2× mais fácil ou mais difícil que o do bloco anterior. Isso suaviza a janela curta do aquecimento e limita o efeito de timestamps manipulados, que já são contidos pela limitação de t_j a $[1, 6T]$ e pela regra da mediana.

O cálculo usa aritmética inteira de 512 bits e o alvo é limitado pelo **pow limit** da rede.

Justiça × eficiência. Um ASIC de SHA-256 é muito mais eficiente em energia por hash do que uma CPU — mas essa eficiência concentra a mineração em quem tem acesso a fábricas de chips e energia subsidiada. The Coin escolhe conscientemente a **justiça**: uma função cujo custo é dominado por memória, de modo que o hardware que já existe em todo lugar (CPUs de servidores e VPS) continue competitivo, e um ajuste de dificuldade que não premia quem entra e sai em rajadas. O preço pago é modesto e medido: ≈ 12 ms de verificação por bloco e 16 MiB por thread de mineração.

5.3 Regras de tempo

- O timestamp deve ser maior que a mediana dos 11 blocos anteriores (**median time past**).
- O timestamp não pode estar mais de 180 s à frente do relógio local do nó.

5.4 Escolha da cadeia e finalidade

Cada nó segue a cadeia válida com **maior trabalho acumulado** ($\sum 2^{256}/(\text{alvo} + 1)$); em empate, a primeira vista. Uma reorganização desconecta blocos usando dados de **undo** e conecta os blocos do novo ramo, tudo em uma única transação atômica do banco de dados — se qualquer bloco do novo ramo for inválido, nada muda.

Como proteção adicional de uma rede em crescimento, reorganizações com mais de **720 blocos** (≈ 12 horas) são recusadas, e checkpoints podem ser fixados no código a partir de alturas já consolidadas (a lista está vazia no lançamento). Isso limita o dano de um ataque de 51% de curta duração a janelas recentes. A finalidade na v0.2 é probabilística; a finalidade econômica por votação de validadores faz parte do plano da Seção 21.

6 Política monetária

6.1 Emissão

A emissão segue o modelo do Bitcoin, comprimido para o horizonte de estabilização desejado:

$$\text{subsídio}(h) = \begin{cases} 0 & \text{se } h = 0 \\ 40 \text{ TCN} \gg \lfloor (h-1)/625\,000 \rfloor & \text{caso contrário} \end{cases}$$

Com blocos de 60 s são produzidos $\approx 525\,960$ blocos por ano, e um halving ocorre a cada ≈ 434 dias (1,19 ano). A soma geométrica limita a oferta:

$$\sum \text{subsídios} < 40 \times 625\,000 \times 2 = 50\,000\,000 \text{ TCN}$$

Por causa do truncamento inteiro dos halvings, o total emitido é 49 999 999,91875 TCN (valor impresso por `thecoin params`) — o limite nunca é ultrapassado, e cada nó rejeita um bloco que faria a emissão acumulada exceder 50 milhões.

Era	Recompensa	Emitido na era	Acumulado	% da oferta	Fim (anos)
1	40,0 TCN	25,000 M	25,000 M	50,00%	1,19
2	20,0 TCN	12,500 M	37,500 M	75,00%	2,38
3	10,0 TCN	6,250 M	43,750 M	87,50%	3,56
4	5,0 TCN	3,125 M	46,875 M	93,75%	4,75
5	2,500 TCN	1,563 M	48,438 M	96,88%	5,94
6	1,250 TCN	0,781 M	49,219 M	98,44%	7,13
7	0,625 TCN	0,391 M	49,609 M	99,22%	8,32
8	0,313 TCN	0,195 M	49,805 M	99,61%	9,51

Tabela 6: Cronograma de emissão. As quatro primeiras eras formam o ciclo de estabilização ($\approx 4,75$ anos, 93,75% da oferta).

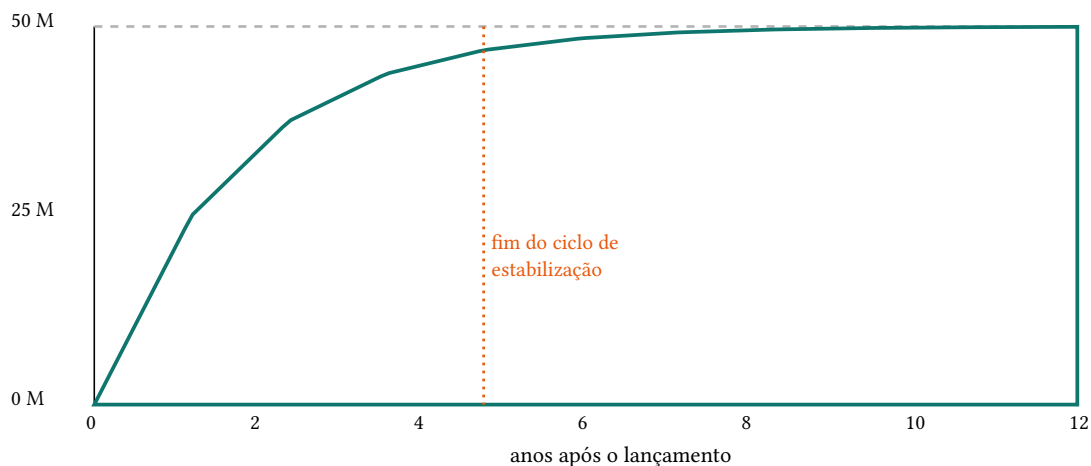


Figura 1: Oferta emitida ao longo do tempo (limite de 50 milhões de TCN).

6.2 Por que um ciclo de cinco anos

O Bitcoin leva mais de uma década para emitir a maior parte da oferta. Uma rede nova e pequena precisa atrair participantes rapidamente e, ao mesmo tempo, chegar logo a uma oferta estável que não dependa de inflação alta. Em $\approx 4,75$ anos, 93,75% das moedas estão distribuídas a quem efetivamente protegeu a rede. A partir daí, a emissão continua caindo pela metade a cada $\approx 1,19$ ano ($\approx 1,56$ milhão de TCN na quinta era) e a segurança passa gradualmente a ser financiada pelas taxas — ou, se a comunidade aprovar, por stake (Seção 21).

6.3 Liberação da recompensa, queima e oferta circulante

- **Liberação escalonada (cooldown).** A recompensa de um bloco (subsídio + taxas do minerador) fica pendente no estado. 25% tornam-se gastáveis após **100 blocos** (`coinbase_maturity`) e os 75% restantes após **1 000 blocos** (`reward_unlock_blocks`, $\approx 16,7$ horas). O prazo final foi escolhido maior que a reorganização máxima (720): recompensas de blocos que ainda poderiam ser revertidos nunca estão totalmente gastáveis (Seção 8).
- **Queima de taxas.** A parte da taxa que corresponde à sobretaxa de congestionamento é destruída (Seção 7).

- **Queima de depósitos.** Depósitos de propostas de governança que não atingem quórum são queimados.
- A oferta circulante é emitido – queimado, contabilizada no registro global do estado.
- Não há pré-mineração, alocação para fundadores ou reserva de desenvolvimento: todo TCN nasce da prova de trabalho.

7 Taxas

7.1 O modelo

Cada transação consome três recursos escassos de todos os nós: **processamento e verificação** (uma assinatura, um registro de conta), **bytes** (banda e disco para sempre, nos nós arquivo) e, se executar contrato, **combustível** (tempo de CPU da máquina virtual). A taxa mínima cobra cada um separadamente e multiplica o total pelo congestionamento atual da rede:

$$\text{taxa}_{\text{mín}} = \left\lceil \left(\text{base_fee} + \left\lceil \text{tamanho} \cdot \frac{\text{fee_per_kb}}{1000} \right\rceil + \left\lceil \text{max_fuel} \cdot \frac{\text{fee_per_kfuel}}{1000} \right\rceil \right) \cdot \frac{\text{congestion_bp}}{10000} \right\rceil$$

onde **tamanho** é o tamanho serializado em bytes, **max_fuel** é o combustível **reservado** pela transação (zero para transações nativas) e **congestion_bp** é o multiplicador em pontos-base ($10\,000 = 1\times$). Tudo é aritmética inteira em moes. Com o multiplicador em $1\times$, a taxa mínima é a **taxa base** B ; a diferença $\text{taxa}_{\text{mín}} - B$ é a **sobretaxa de congestionamento**.

Destino	Valor	Por quê
Queimado	$\text{taxa}_{\text{mín}} - B$	Mineradores não lucram lotando blocos com transações próprias para empurrar as taxas para cima: a sobretaxa que eles próprios pagariam seria destruída.
Minerador	$\text{fee} - (\text{taxa}_{\text{mín}} - B)$	A taxa base mais toda a gorjeta (o que se paga acima do mínimo) remuneram quem inclui a transação.

Tabela 7: Divisão da taxa paga (**fee**). O recibo de cada transação registra **fee** e **burned**.

7.2 Multiplicador de congestionamento

Ao final de cada bloco, o nó mede a ocupação do bloco — o **maior** entre a fração de bytes usada ($\text{tamanho} / \text{max_block_bytes}$) e a fração de combustível reservada ($\text{combustível} / \text{max_block_fuel}$) — e ajusta o multiplicador para o bloco seguinte, mirando **50%** de ocupação:

$$c_{h+1} = \text{limitar} \left(c_h \pm \left\lfloor \frac{c_h \cdot |o - 5000|}{8 \cdot 5000} \right\rfloor, 1\times, 1000\times \right)$$

com a ocupação o em pontos-base (0 a 10 000). Um bloco totalmente cheio aumenta o multiplicador em **12,5%**; um bloco vazio o reduz em 12,5%; ocupações intermediárias movem proporcionalmente, e 50% exatos o mantêm. A faixa é de $1\times$ ($\text{CONGESTION_MIN_BP} = 10\,000$) a $1\,000\times$ ($\text{CONGESTION_MAX_BP} = 10\,000\,000$). O mecanismo é da mesma família do EIP-1559 do Ethereum [12], adaptado para dois recursos e com a taxa base (não só a sobretaxa) paga ao minerador.

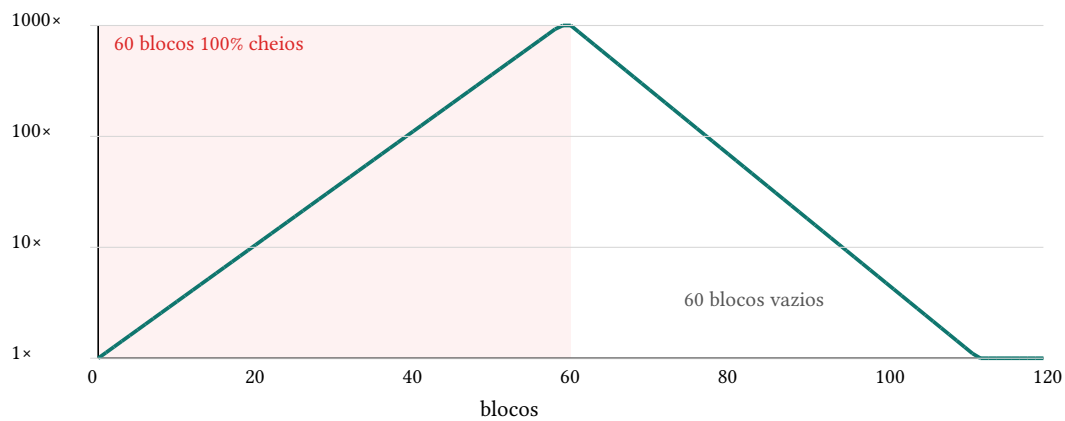


Figura 2: Multiplicador de congestionamento (escala logarítmica) simulado com a regra de consenso: 6 blocos cheios dobram a taxa, 20 a multiplicam por 10, 59 atingem o teto de 1000x; 52 blocos vazios a trazem de volta a 1x.

7.3 Por que este modelo

- **Protege contra inundação.** Um atacante que tenta ocupar a rede com transações próprias paga uma taxa que cresce 12,5% a cada bloco cheio — em uma hora de blocos cheios, cada transação custa 1000x mais — e a sobretaxa é queimada, então nem um minerador atacante recupera o gasto.
- **Favorece usuários reais.** Quem precisa pagar agora paga gorjeta e é priorizado (Seção 7.5); quem pode esperar aguarda poucos blocos até o multiplicador cair. Transações que ficaram abaixo do mínimo por causa de uma alta de congestionamento **permanecem** no mempool e voltam a ser válidas quando a taxa cai, em vez de serem descartadas.
- **Cobra o recurso certo.** Uma transferência não paga pelo combustível de contratos; um contrato pesado não se esconde atrás de poucos bytes.
- **Continua barata.** Os valores são expressos em moedas e são parâmetros de governança com limites rígidos (Seção 14): se o TCN se valorizar, detentores e mineradores podem reduzir `base_fee`, `fee_per_kb` e `fee_per_kfuel` sem nenhuma atualização de software.

7.4 Exemplos com os parâmetros padrão da mainnet

Padrões: `base_fee` = 1000 moedas (0,00001 TCN), `fee_per_kb` = 10000 moedas (10 moedas por byte), `fee_per_kfuel` = 1000 moedas por 1000 de combustível, `storage_deposit_per_kb` = 100000 moedas (0,001 TCN). Os valores abaixo são calculados neste documento pelas mesmas fórmulas inteiras do código.

Congestionamento	Taxa mínima	Queimado	Taxa normal (1,25x)	Minerador recebe
1x	2 590 moedas	0	3 238 (0,00003238 TCN)	3 238
2x	5 180 moedas	2 590	6 475 (0,00006475 TCN)	3 885
10x	25 900 moedas	23 310	32 375 (0,00032375 TCN)	9 065
1000x	2 590 000 moedas	2 587 410	3 237 500 (0,032375 TCN)	650 090

Tabela 8: Transferência simples de 159 bytes: $B = 1000 + \lceil 159 \times 10000/1000 \rceil = 2590$ moedas = 0,0000259 TCN.

Outros casos a 1x de congestionamento:

- Transferência com memo de 32 bytes (191 bytes): 2910 moedas.
- Pagamento em lote para 128 destinatários (3719 bytes): 38190 moedas no total, ≈ 298 moedas por destinatário.
- **Publicação de contrato.** O contrato de exemplo `token.tccl` (1701 bytes de código-fonte) gera uma transação `Deploy` de **1860 bytes**. Compilar custa 5 de combustível por byte de fonte (8505) e o `init` consome 995: **9500 de combustível** medidos. A carteira reserva o medido + 30% + 5000, ou seja, `max_fuel` = 17350. Taxa mínima: $1000 + 18600 + 17350 = 36950$ moedas (0,0003695 TCN); com prioridade `normal`, 46188 moedas (0,00046188 TCN).

- **Depósito de armazenamento do mesmo contrato.** Código compilado + armazenamento inicial = 1311 bytes $\rightarrow \lceil 1311/1000 \rceil \times 100000 = 200000$ motes = 0,002 TCN, **devolvidos** quando o armazenamento diminui ou o contrato é destruído.

Conferência em regtest. A rede local de testes usa parâmetros 10× menores (`base_fee` 100, `fee_per_kb` 1000, `fee_per_kfuel` 100, depósito 10 000 por kB). A mesma publicação medida em regtest custou 4619 motes (0,00004619 TCN) com prioridade `normal` e depósito de 0,0002 TCN — exatamente o que as fórmulas acima preveem.



Figura 3: Composição da taxa de publicação do `token.tccl` com congestionamento em 2× e prioridade `normal`.

7.5 Prioridade: pagar mais para confirmar antes

Mineradores montam blocos escolhendo primeiro as transações com maior **taxa por unidade de peso**, respeitando a ordem de nonce de cada remetente:

$$\text{peso} = \text{tamanho em bytes} + \frac{\text{max_fuel}}{100} \quad \text{prioridade} = \frac{\text{fee}}{\text{peso}}$$

(100 unidades de combustível equivalem a 1 byte.) A carteira oferece quatro níveis, calculados pelo nó a partir do mempool (`GET /api/v1/fees`) e aplicados como multiplicador sobre a taxa mínima atual:

Nível	Multiplicador	Quando usar
<code>low</code>	1× (mínimo)	Blocos não estão cheios; não há pressa.
<code>normal</code> (padrão)	1,25×	Continua válida mesmo se o congestionamento subir no bloco seguinte (alta máxima de 12,5% por bloco).
<code>high</code>	$\geq 2\times$ — taxa por peso necessária para caber em um bloco inteiro de transações pendentes, + 0,1×	Passar à frente da maioria das transações em espera.
<code>urgent</code>	$\geq 2 \times \text{high}$ — taxa para ficar no primeiro quarto do bloco, + 0,25×	Próximo bloco com probabilidade muito alta.

Tabela 9: Níveis de prioridade (`thecoin-wallet --priority low|normal|high|urgent`).

Com o mempool vazio, `high` = 2× e `urgent` = 4× a taxa mínima. Numa transferência típica a 1×, isso significa 0,0000259 / 0,0000324 / 0,0000518 / 0,0001036 TCN. O comando `thecoin-wallet fees` mostra os valores atuais.

8 Confirmação rápida e segura

Pagamentos do dia a dia precisam ser aceitos em segundos; pagamentos grandes precisam ser irreversíveis. A v0.2 ataca os dois lados.

8.1 Substituição por taxa opcional

Uma transação enviada com a flag `replaceable` (`--replaceable` na carteira) pode ser trocada no mempool por outra do mesmo remetente e nonce com taxa **pelo menos 25% maior** (como no BIP-125 [13]; `thecoin-wallet bump-fee <txid>`). Sem a flag, a primeira versão vista é definitiva para os nós: uma segunda transação com o mesmo nonce é recusada. Quem recebe um pagamento vê a flag e sabe se pode confiar na transação ainda não confirmada ou se deve esperar confirmação — o contrário do RBF implícito da v0.1, em que todo pagamento pendente podia ser substituído.

8.2 Alertas de gasto duplo

Quando um nó recebe uma segunda transação, com o mesmo remetente e nonce de uma pendente não substituível, ele registra a tentativa e envia a mensagem P2P `DoubleSpend` com **as duas transações assinadas**. Cada nó que a recebe verifica que ambas são válidas e do mesmo remetente e nonce — a prova não pode ser forjada sem a chave do remetente — e a repassa. Em segundos toda a rede sabe da tentativa: lojistas consultam `GET /api/v1/alerts` (ou `thecoin-wallet alerts`), e a consulta da transação passa a indicar o conflito. Cada nó guarda até 512 alertas recentes.

8.3 Blocos compactos

Um novo bloco é anunciado como **bloco compacto** (no estilo do BIP-152 [14]): cabeçalho + um identificador curto de 6 bytes por transação, calculado com o hash do bloco como sal (colisões não podem ser pré-calculadas). O receptor reconstrói o bloco com as transações que já tem no mempool e pede apenas as que faltam (`GetBlockTx` → `BlockTx`); em caso de divergência, baixa o bloco completo. Um bloco cheio de 6 288 transferências (≈ 1 MB) é anunciado com ≈ 38 kB. O nó valida o cabeçalho e a prova de trabalho de um bloco compacto antes de alocar qualquer coisa para suas transações.

Por que isso importa para a segurança: propagação mais rápida reduz o tempo em que dois mineradores encontram blocos concorrentes sem saber um do outro — menos blocos órfãos, menos reorganizações de 1 bloco e menos vantagem para mineradores com melhor conectividade.

8.4 Quantas confirmações esperar

Reverter um pagamento com N confirmações exige que o atacante refaça a prova de trabalho desses N blocos mais rápido que o resto da rede e abra mão das recompensas que a cadeia honesta pagaria. O nó recomenda o menor N tal que o valor em jogo seja pelo menos o **dobro** do pagamento:

$$N = \text{limitar} \left(\left\lceil 2 \cdot \frac{\text{valor}}{\text{subsídio do próximo bloco}} \right\rceil, 1, 720 \right)$$

disponível em `GET /api/v1/security?amount=<valor>` e `thecoin-wallet confirmations <valor>`.

Valor do pagamento	Confirmações (era 1, 40 TCN)	Tempo aproximado	Era 2 (20 TCN)
1 TCN	1	1 min	1
20 TCN	1	1 min	2
100 TCN	5	5 min	10
1 000 TCN	50	50 min	100
5 000 TCN	250	4,2 h	500
10 000 TCN	500	8,3 h	720
20 000 TCN	720	12,0 h	720

Tabela 10: Confirmações recomendadas pelo estimador. Acima de 720 blocos (≈ 12 h) a cadeia nunca reorganiza.

8.5 Reorganização máxima e liberação escalonada da recompensa

Duas regras de consenso completam a proteção:

- **Reorganização máxima de 720 blocos.** Nenhum nó abandona mais de 720 blocos de sua cadeia, qualquer que seja o trabalho do ramo concorrente. Um pagamento com 720 confirmações é final para todos os nós que o viram.
- **Cooldown da recompensa (Seção 6.3).** Só 25% da recompensa de um bloco é gastável após 100 blocos; o resto só após 1000, depois do limite de reorganização. Um minerador que tenta reescrever a cadeia arrisca recompensas ainda bloqueadas, e não consegue vender imediatamente o que minera para financiar ou lucrar com o ataque.

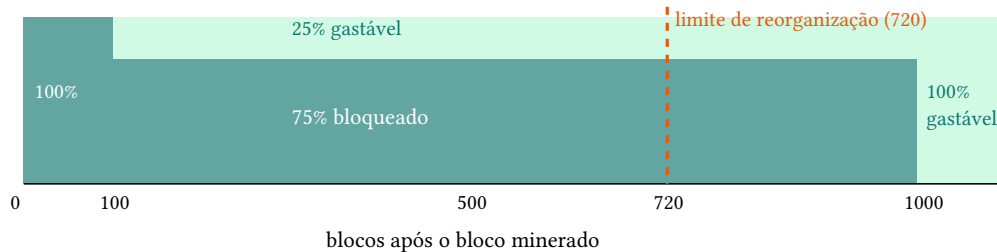


Figura 4: Liberação da recompensa de um bloco na mainnet (subsídio + taxas do minerador); escuro = bloqueado, claro = gastável.

9 Estado global e armazenamento compacto

9.1 Registros de estado

O estado é um mapa chave-valor com prefixos: contas (saldo, nonce, bloqueio de votos), contratos nativos, propostas, votos, recompensas pendentes, o registro global (emissão, queima, parâmetros de governança, propostas em votação e o multiplicador de congestionamento) e, desde a v0.2, três prefixos para contratos TCCL: metadados, código compilado e armazenamento. Contas vazias não são armazenadas; contratos concluídos ou destruídos são apagados. O estado cresce com o número de contas e contratos ativos, não com o histórico — e o armazenamento de contratos é coberto por depósito.

9.2 Compromisso homomórfico de estado (LtHash)

Todo cabeçalho contém a raiz do estado após o bloco. Recalcular uma árvore Merkle de todo o estado a cada bloco seria caro para máquinas pequenas. The Coin usa **LtHash16**: cada registro (k, v) é expandido por BLAKE3-XOF em um vetor de 2048 inteiros de 16 bits, e o compromisso é a soma, módulo 2^{16} , dos vetores de todos os registros:

$$L(\text{estado}) = \sum_{(k,v) \in \text{estado}} \text{XOF}(k, v) \bmod 2^{16}, \quad \text{state_root} = H_{\text{state-root}(L)}$$

Inserir ou remover um registro é somar ou subtrair um vetor, então atualizar o compromisso custa proporcionalmente aos registros alterados pelo bloco. A segurança se apoia no problema de reticulados **Short Integer Solution**. Qualquer divergência — bug, corrupção de disco ou manipulação — faz o nó rejeitar o bloco imediatamente, e ao iniciar o nó confere que o compromisso salvo corresponde ao topo.

9.3 Banco de dados (esquema 2)

Tabela	Chave	Valor
headers, main	hash / altura	Cabeçalhos com trabalho acumulado; cadeia principal.
blocks	hash	Corpo do bloco, zstd.
state	chave de estado	Registro de estado (Borsh).

Tabela	Chave	Valor
undo	altura	Dados para reorganização, zstd; só os últimos 736 blocos.
txindex	txid	Altura (8) + posição (4).
addrindex	endereço + altura + posição	Vazio na v0.2 (a v0.1 guardava o txid de 32 bytes).
receipts	txid	Novo: recibo da transação, zstd.
meta	nome	Topo, compromisso de estado, versão do esquema.

Tabela 11: Tabelas do banco `chain.redb`.

- **redb:** banco embarcado em Rust puro, com transações ACID e árvores B copy-on-write. Cada bloco — estado, undo, índices, recibos, topo e compromisso — é gravado em **uma** transação: uma queda de energia nunca deixa um bloco pela metade.
- **Recibos:** para cada transação confirmada o nó guarda taxa, parte queimada, endereços afetados, contrato criado, sucesso ou erro, combustível usado, eventos emitidos e valor de retorno. Exploradores e carteiras mostram o resultado real de cada transação sem reexecutá-la.
- **Índice de endereços compacto:** a entrada é apenas a chave ordenada; o txid é lido do bloco na consulta. Isso eliminou 32 bytes por par (endereço, transação).
- **Podar:** nós podados mantêm os últimos `prune_keep` corpos de blocos (mínimo 1000), cabeçalhos e estado. Ao podar um bloco o nó remove também suas entradas no índice de transações e seus recibos, e nós podados não mantêm o índice de endereços, que apontaria para corpos apagados.
- **Mempool persistente:** transações pendentes sobrevivem a reinicializações (`mempool.dat`).
- **Compactação:** `thecoind compact` (com o nó parado) devolve ao sistema o espaço pré-alocado.
- **Cache configurável** (64 MB por padrão).

Os números medidos e as projeções de disco estão na Seção 20.

10 Transações e pagamentos

- **Transferência** com **memo** de até 256 bytes (número de pedido, fatura, mensagem).
- **Pagamento em lote** para até 128 destinatários em uma única transação (folha de pagamento, pools, exchanges).
- **Validade** opcional (`expiry_height`) para que cobranças antigas não sejam executadas tarde demais.
- **Prioridade** e **substituição por taxa opcional** (Seção 7.5, Seção 8).
- **Simulação antes de assinar:** `POST /api/v1/tx/simulate` executa a transação sobre o estado atual e devolve validade, taxa mínima, combustível usado, eventos e retorno — a carteira usa isso para medir o combustível e para nunca enviar uma chamada de contrato que falharia.
- **Solicitações de pagamento** padronizadas por URI, prontas para QR code:

```
thecoin:tc1q...?amount=12.5&memo=Pedido%20%23123&label=Loja
```

11 Contratos de pagamento nativos

Para os casos mais comuns, a rede oferece **modelos nativos de contratos de pagamento**. Eles não são Turing-completos, não têm combustível, laços ou reentrância, e cada chamada tem custo fixo — portanto são seguros e baratos de validar em máquinas pequenas. Os fundos ficam no registro do contrato e só saem pelas regras abaixo.

Modelo	Regras	Casos de uso
Escrow	Pagador bloqueia o valor. Pagador ou árbitro liberam ao recebedor; recebedor ou árbitro	Comércio eletrônico, serviços, compra entre desconhecidos.

Modelo	Regras	Casos de uso
	reembolsam; após o prazo o pagador reembolsa sozinho.	
Vesting	Libera linearmente entre início e fim, nada antes do cliff . Se revogável, o criador recupera a parte não liberada.	Salários, bônus, distribuição a equipes.
Assinatura	Pagamento pré-pago de N períodos; o recebedor saca um período imediatamente e mais um a cada <code>period_blocks</code> ; o pagador cancela e recupera os períodos futuros.	SaaS, mensalidades, aluguel.
HTLC	Paga ao destinatário quem revelar a pré-imagem com SHA-256 igual ao hash lock até o prazo; depois, o remetente recupera.	Trocas atômicas com Bitcoin e outras redes, canais de pagamento.
Multisig	Cofre controlado por M de N signatários (até 16); gastos propostos e aprovados on-chain. Novo: <code>MultisigClose</code> apaga um cofre vazio.	Tesouraria de empresas e associações, custódia compartilhada.

Tabela 12: Modelos nativos de contratos de pagamento.

O identificador de um contrato nativo é $H_{\text{contract-id}}(\text{criador} \parallel \text{nonce})$, conhecido antes mesmo da confirmação. Na v0.2, criar um contrato nativo também trava um **depósito de armazenamento** proporcional ao tamanho do registro (`storage_deposit_per_kb`), devolvido ao criador quando o contrato termina (saldo zerado) ou, no caso do multisig, quando é fechado por um signatário.

12 Contratos inteligentes TCCL

A v0.1 previa uma VM WebAssembly para contratos gerais. A v0.2 entrega, em vez disso, uma linguagem própria, pequena e verificável: **TCCL — The Coin Cloud Language**. A escolha é deliberada: uma linguagem de alto nível com regras estritas e uma VM interpretada simples são mais fáceis de auditar e de reimplementar do que um motor WebAssembly completo, e cabem no orçamento de máquinas modestas.

12.1 Princípios de projeto

- **Tipos estritos.** Toda constante, variável de estado, parâmetro e variável local declara o tipo (`int`, `bool`, `text`, `bytes`, `address`, `list[T]`, `map[K, V]`). Cada expressão tem exatamente um tipo; não há conversões implícitas; funções com retorno retornam em todos os caminhos. `int` é um inteiro de 128 bits com toda aritmética verificada: estouro aborta a chamada.
- **Funções com papéis explícitos.** `init` roda na publicação; `action` muda estado e é chamada por transação; `view` é somente leitura, gratuita pela API, e o compilador garante que não altera estado, não envia TCN nem emite eventos; `fn` é auxiliar interna. Só funções `payable` recebem TCN.
- **Determinismo.** Não há ponto flutuante, relógio nem aleatoriedade; o contexto é só `caller`, `value`, `balance`, `height` e `self`. A mesma chamada sobre o mesmo estado dá o mesmo resultado em todos os nós.
- **Sem chamadas entre contratos.** Um contrato pode enviar TCN a qualquer endereço, mas não chama outro contrato — reentrância é impossível por construção.
- **O compilador faz parte do consenso.** A transação `Deploy` carrega o **código-fonte**; cada nó o compila de forma idêntica e guarda o programa compilado. O código-fonte fica na cadeia para sempre (o contrato registra o txid de publicação e o hash BLAKE3 do fonte): qualquer pessoa pode ler e verificar exatamente o que um contrato faz.
- **Combustível.** Cada operação consome combustível; a transação declara `max_fuel` e paga por ele (Seção 7). Acabou o combustível, a chamada aborta.

- **Falha paga taxa e reverte.** Toda execução roda numa camada de estado filha. Se o contrato falhar (**require** falso, falta de combustível, estouro, erro de compilação), essa camada é descartada: o valor enviado, as escritas, os envios e os eventos são revertidos; só a taxa é cobrada e o recibo registra o erro. O mempool simula cada chamada e **recusa** as que falhariam, para que o usuário não pague por um erro previsível.

12.2 Exemplo

Contrato completo `crates/tccl/examples/tip_jar.tccl`, reproduzido literalmente:

```
# A tip jar: anyone can send TCN with a message, only the owner withdraws.
contract TipJar

state owner: address
state total_received: int
state tips: int

event Tip(from: address, amount: int, message: text)
event Withdrawn(to: address, amount: int)

init():
    owner = caller

action tip(message: text) payable:
    require value >= TCN / 100, "minimum tip is 0.01 TCN"
    require len(message) <= 140, "message too long"
    total_received += value
    tips += 1
    emit Tip(caller, value, message)

action withdraw(amount: int):
    require caller == owner, "only the owner can withdraw"
    require amount > 0 and amount <= balance, "invalid amount"
    send(owner, amount)
    emit Withdrawn(owner, amount)

view stats() -> list[int]:
    return [total_received, tips, balance]
```

Fluxo típico, do simulador local à rede:

```
tccl check tip_jar.tccl           # compila e mostra a interface
tccl run tip_jar.tccl deploy      # simulador local com contas de teste
thecoin-wallet contract deploy tip_jar.tccl
thecoin-wallet contract invoke <endereço> tip "obrigado!" --value 0.05
thecoin-wallet contract view <endereço> stats
```

O repositório traz ainda `counter`, `token`, `crowdfund` e `private_pool`. O guia de receitas em inglês **TCCL Cookbook** (`docs/tccl/tccl-cookbook.pdf`) cobre a linguagem em detalhe.

12.3 Recibos e eventos

Cada transação confirmada gera um **recibo** (Seção 9) com sucesso ou erro, combustível usado, valor de retorno e os **eventos** emitidos (`emit`, até 64 por chamada), cada um com o endereço do contrato, o nome e os campos tipados. Eventos são a interface para carteiras, exploradores e aplicações: um lojista acompanha `Tip` sem ler o armazenamento do contrato.

12.4 Custos e limites

Operação	Combustível
Instrução / expressão	2 / 1
Chamada de função	20

Limite	Valor
Código-fonte	48 000 bytes
Transação <code>Deploy</code>	64 000 bytes
Programa compilado	262 144 bytes

Operação	Combustível
Leitura de armazenamento	250
Escrita	400 + 4/byte
Hash (SHA-256, BLAKE3)	60 + 20/64 bytes
Verificar Ed25519	3 500
Verificar assinatura em anel	5 000 + 10 000/membro
<code>send</code> / <code>emit</code> / <code>destroy</code>	300 / 100 / 1000
Compilar (publicação)	5/byte de fonte
Carregar contrato (chamada)	100 + 1/100 bytes

Tabela 13: Tabela de combustível (consenso).

A taxa é calculada sobre o combustível **reservado** (`max_fuel`), que também conta para o limite do bloco; por isso a carteira mede o consumo real por simulação e reserva só uma margem.

Os preços foram **calibrados por medição** (`crates/tccl/tests/fuel_bench.rs` e `crates/node/tests/fuel_storage_bench.rs`, com 400 000 entradas reais no banco): cada operação custa cerca de 20 ns de CPU por unidade de combustível numa VPS de 2 vCPU. Assim, mesmo um bloco **completamente cheio** (50 milhões) com a pior operação possível executa em cerca de 1,2 s — pouco diante dos 60 s de intervalo — e ninguém consegue produzir blocos lentos de validar sem pagar por isso.

12.5 Depósito de armazenamento reembolsável

Estado que fica para sempre em todos os nós tem um custo que uma taxa única não cobre. Por isso todo contrato mantém um **depósito** de `storage_deposit_per_kb` por kB de estado (código compilado + chaves e valores do armazenamento), arredondado para cima em kB:

- Quem faz o estado **crescer** paga a diferença, até o limite `max_deposit` que declarou na transação (se exceder, a chamada falha e é revertida).
- Quem faz o estado **diminuir** recebe de volta a parte proporcional: $\text{reembolso} = \text{depósito} \cdot \frac{\text{bytes}_{\text{antes}} - \text{bytes}_{\text{depois}}}{\text{bytes}_{\text{antes}}}$.
- `destroy(to)` apaga o contrato e envia a `to` o saldo restante e **todo o depósito**.

O efeito é um incentivo econômico para limpar estado — apagar entradas de um mapa devolve dinheiro — e uma garantia de que o crescimento do estado é sempre pago por quem o causa, não pelos operadores de nós.

12.6 Liberdade total para desenvolvedores

Publicar um contrato não exige permissão, lista de aprovação ou taxa especial: qualquer código que o compilador aceite pode ser publicado por qualquer pessoa, e qualquer pessoa pode chamá-lo. O protocolo não tem chaves de administrador sobre contratos; as regras de cada contrato são as que estão no seu código-fonte público. Tokens, financiamento coletivo, cofres, jogos, votações e pools de privacidade são aplicações, não mudanças de protocolo. Os limites existem apenas para proteger os nós — combustível, tamanho e depósito — e são iguais para todos.

Limite	Valor
Combustível por transação	10 000 000
Combustível por bloco (padrão)	50 000 000
Argumentos por chamada	32
Eventos por chamada	64
Tamanho de um valor	64 KiB
Itens de lista em memória	4 096
Profundidade de chamadas	16
Membros de um anel	64
Operadores encadeados / profundidade	64 / 128
Aninhamento de blocos e tipos	32

Tabela 14: Limites da linguagem e da VM.

13 Privacidade opcional por contratos

A camada base da The Coin é **transparente**: saldos, remetentes, destinatários e valores são públicos, como no Bitcoin, o que facilita auditoria, conformidade e verificação da oferta. A privacidade é **opcional** e construída em contratos, com uma primitiva criptográfica disponível na TCCL.

13.1 Assinaturas em anel (bLSAG)

`ring_verify(anel, mensagem, assinatura, key_image)` verifica uma assinatura em anel ligável no esquema **bLSAG** (Back's Linkable Spontaneous Anonymous Group, como no Monero [15]), sobre o grupo de ordem prima **Ristretto255** [16] (sem as armadilhas de cofator da curva 25519), com todos os hashes separados por domínio (`crates/tccl/src/ring.rs`):

- A assinatura prova “o signatário possui a chave secreta de **um** destes n membros” sem revelar qual. Ela tem $32(n + 1)$ bytes e está ligada à mensagem e ao conjunto exato de membros.
- A **key image** $I = x \cdot H_{p(P)}$ é única por chave secreta: a mesma chave produz sempre a mesma imagem, em qualquer anel. Um contrato que guarda as imagens usadas impede que a mesma chave assine duas vezes — sem que a imagem revele qual membro a produziu.

13.2 Pool de valor fixo

O exemplo `crates/tccl/examples/private_pool.tccl` implementa a interface padrão **TCCL-PRIV-1** usada pela carteira:

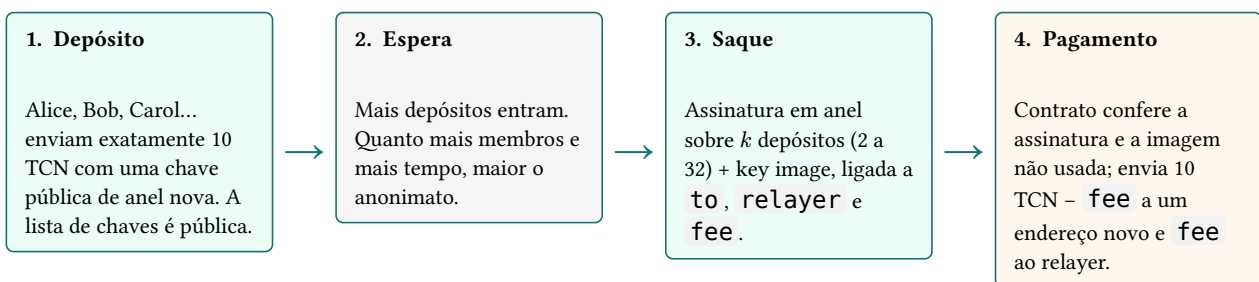


Figura 5: Fluxo do pool de privacidade de valor fixo.

Trecho literal da função de saque:

```

action withdraw(to: address, relayer: address, fee: int, members: list[int], signature: bytes, key_image: bytes):
    require len(members) >= MIN_RING and len(members) <= MAX_RING, "ring size must be 2 to 32"
    require fee >= 0 and fee < DENOMINATION, "invalid relayer fee"
    require not used.has(key_image), "this deposit was already withdrawn"
    let ring: list[bytes] = []
    for i in members:
        require i >= 0 and i < len(keys), "unknown deposit index"
        ring.push(keys[i])
    let message: bytes = withdraw_message(to, relayer, fee)
    require ring_verify(ring, message, signature, key_image), "invalid ring signature"
    used[key_image] = true
    withdrawn += 1
    send(to, DENOMINATION - fee)
    if fee > 0:
        send(relayer, fee)
    emit Withdrawn(len(members))
  
```

Valor fixo elimina a correlação por valor: todo depósito e todo saque são de 10 TCN. **Relayers** resolvem o problema do endereço novo sem saldo: qualquer terceiro pode enviar a transação de saque (pagando a taxa de rede) e receber **fee** do próprio saque. Como a mensagem assinada inclui **to**, **relayer** e **fee**, o relayer não pode desviar o valor nem alterar sua remuneração. Na carteira: `thecoin-wallet privacy keygen | deposit | withdraw | status`; as chaves de anel derivam da mesma frase de recuperação, e o saque escolhe membros aleatórios (16 por padrão).

13.3 Limites do anonimato

- **O conjunto de anonimato é o anel, não o pool.** Um saque com anel de 16 esconde o depositante entre 16 depósitos — não entre todos. Use anéis maiores (até 32 neste pool) quando o pool permitir.
- **Correlação por tempo.** Sacar logo após depositar, quando há poucos depósitos novos, reduz o anonimato na prática. Espere o pool crescer e evite padrões (sempre à mesma hora, logo após receber).
- **Quem envia a transação de saque é público.** Se você enviar o saque do mesmo endereço que depositou, a ligação é óbvia; a carteira avisa. Use um relayer ou um endereço sem relação.
- **Metadados fora da cadeia.** O IP do nó que primeiro retransmite a transação, o uso posterior dos fundos junto com fundos identificados e a reutilização de endereços podem revelar a ligação.
- **Depósitos e valores continuam visíveis.** O pool esconde **qual** depósito corresponde a **qual** saque; não esconde que alguém usou o pool.

A camada base permanece transparente e auditável; a privacidade é uma escolha do usuário, com custo e limites conhecidos, e novas construções (outros valores, outros esquemas) podem surgir como contratos sem mudar o protocolo.

14 Governança bicameral

The Coin foi pensada para evoluir sem donos. Mudanças de parâmetros são decididas on-chain por um sistema **bicameral**: uma proposta só passa se as duas câmaras aprovarem.

1. **Câmara dos detentores.** Qualquer endereço vota com um peso em TCN. As moedas usadas no voto ficam **bloqueadas** até o fim da votação e não podem ser movidas — portanto não podem ser transferidas para outro endereço e usadas de novo. Cada endereço vota uma vez por proposta.
2. **Câmara dos mineradores.** Cada proposta em votação recebe um bit do campo `signal` (até 32 propostas simultâneas); mineradores que apoiam a proposta ligam o bit nos blocos que produzem.

Critério	Regra (em pontos-base, 10 000 = 100%)
Quórum	votos totais (sim + não + abstenção) $\geq$ <code>quorum_bp</code> $\times$ oferta circulante
Aprovação dos detentores	sim $\geq$ <code>approval_bp</code> $\times$ (sim + não)
Aprovação dos mineradores	blocos sinalizando $\geq$ <code>miner_approval_bp</code> $\times$ blocos da votação
Duração	<code>vote_period</code> blocos
Ativação	<code>activation_delay</code> blocos após aprovação
Depósito	<code>proposal_deposit</code> , devolvido se houver quórum, queimado caso contrário

Tabela 15: Regras de aprovação de propostas.

Ciclo de vida: Votação $\rightarrow$ ao fim, Aprovada ou Rejeitada $\rightarrow$ após o atraso, Ativada. O atraso dá tempo para operadores de nós atualizarem o software quando necessário.

14.1 Parâmetros governáveis

Parâmetro	Padrão mainnet	Limites rígidos (mín.–máx.)
<code>max_block_bytes</code>	1 000 000 (1 MB)	250 000 – 8 000 000
<code>max_block_fuel</code>	50 000 000	5 000 000 – 500 000 000
<code>base_fee</code>	1 000 motes (0,00001 TCN)	0 – 10 000 000
<code>fee_per_kb</code>	10 000 motes/kB	1 – 100 000 000
<code>fee_per_kfuel</code>	1 000 motes/1 000 comb.	1 – 100 000 000
<code>storage_deposit_per_kb</code>	100 000 motes/kB (0,001 TCN)	0 – 1 000 000 000

Parâmetro	Padrão mainnet	Limites rígidos (mín.–máx.)
proposal_deposit	100 TCN	1 – 1 000 000 TCN
vote_period	20 160 blocos (≈ 14 dias)	1 440 – 201 600
quorum_bp	1 000 (10%)	100 – 5 000 (1% – 50%)
approval_bp	6 667 (66,67%)	5 001 – 9 500
miner_approval_bp	6 000 (60%)	5 000 – 9 500
activation_delay	2 880 blocos (≈ 2 dias)	720 – 43 200

Tabela 16: Os 12 parâmetros de governança da v0.2 (`crates/core/src/params.rs`). Testnet usa os mesmos limites; seus padrões diferem em depósito (10 TCN), votação (1 440), quórum (5%), mineradores (50%) e ativação (720).

Uma proposta `SetParam` com valor fora dos limites é recusada na criação, e o valor é conferido de novo na ativação. Propostas de **texto** registram decisões da comunidade, e propostas de **atualização de software** registram a versão aprovada e o hash da release; nós desatualizados passam a exibir um aviso. É por esse mecanismo que as fases do plano de consenso seriam ativadas (Seção 21).

14.2 Sinalização de mineradores

Um minerador declara quais propostas apoia pelo identificador (64 caracteres hexadecimais):

```
thecoind --signal <id>      # repetível (ou mining.signal no thecoind.toml)
thecoin signal <id>         # nó instalado: grava a configuração e reinicia
thecoin unsignal <id>       #
thecoin signals              # propostas abertas e as que seus blocos apoiam
```

O nó liga o bit correspondente apenas enquanto a proposta está em votação; bits não atribuídos tornam o bloco inválido.

O que nunca pode mudar por votação: o limite de 50 milhões de TCN, o cronograma de emissão, o algoritmo de prova de trabalho e as regras de validade de transações. Alterá-los exigiria que cada operador instalasse voluntariamente um software diferente — exatamente como no Bitcoin.

Nenhum grupo decide sozinho: grandes detentores não mudam a rede sem os mineradores, e grandes mineradores não mudam a rede sem os detentores. O site `the-coin.cloud` exibe cada proposta, o andamento das duas câmaras, o quórum e o tempo restante, lendo diretamente a API dos nós.

15 Rede ponto a ponto

- **Transporte:** TCP. Cada mensagem é um **frame** com magic da rede (4 bytes), tamanho, checksum e payload Borsh; frames acima de 9 MiB ou malformados derrubam a conexão.
- **Handshake:** troca de `Version` (gênese, altura, serviços, nonce anti-autoconexão) e `Verack`. Nós de outra rede ou gênese são desconectados.
- **Sincronização:** localizadores de blocos (densos no topo, esparsos para trás) → até 500 hashes por `Inv` → `GetData` → blocos. A prova de trabalho dos blocos recebidos é verificada em paralelo e os blocos são aplicados em ordem por uma thread dedicada.
- **Propagação:** transações por `Inv` apenas a quem ainda não as conhece; novos blocos como **blocos compactos** (`CompactBlock`, `GetBlockTxs`, `BlockTxs`); o mempool de um novo par é solicitado na conexão.
- **Alertas:** `DoubleSpend` carrega duas transações conflitantes assinadas; é verificado e repassado uma única vez por par de transações.
- **Descoberta:** seeds DNS (`seed1.the-coin.cloud`, `seed2.the-coin.cloud`), troca de endereços e um gerenciador de endereços persistente com **back-off** exponencial.

- **Proteção contra abuso:** limites de conexões de entrada (32) e por IP, filas de escrita limitadas (pares lentos são desconectados), pontuação de mau comportamento com banimento de 24 h, limites estruturais de mensagens (até 65 536 IDs por bloco compacto), timeouts de handshake e ping.

16 Nó, mineração e instalação

16.1 Modelo de execução

O nó foi desenhado para não travar a máquina nem a si mesmo:

- rede e API usam um runtime assíncrono Tokio com apenas 2 threads;
- blocos são validados por **uma** thread dedicada, com fila, isolando trabalho pesado da rede;
- leituras (API, P2P) usam transações MVCC do banco e não bloqueiam a escrita;
- threads de mineração rodam com prioridade mínima e, por padrão, usam todos os núcleos menos um;
- o minerador monta o bloco pela ordem de prioridade do mempool, respeitando os limites de bytes e de combustível, e reconstrói o bloco-modelo a cada novo topo ou lote de transações;
- a mineração pausa enquanto o nó sincroniza;
- a API REST tem limite de concorrência, de corpo (256 KiB) e de tempo por requisição.

16.2 Virar validador com uma linha

```
curl -fsSL https://the-coin.cloud/install.sh | sudo bash -s -- --yes
```

Sem `--yes`, o instalador é interativo e pergunta antes de criar a carteira. Ele:

1. faz verificações prévias: sistema e arquitetura (x86_64 ou ARM64), systemd, RAM e disco, sincronização do relógio e firewall; oferece criar 1 GB de swap em máquinas com menos de 2 GB sem swap;
2. baixa `thecoind`, `thecoin-wallet` e `tccl` e **verifica o SHA-256**, recorrendo à compilação do código-fonte quando não há binário;
3. cria o usuário de sistema sem privilégios `thecoin` e `/var/lib/thecoin`;
4. cria a carteira que receberá as recompensas (ou usa `--miner-address`). No modo não interativo, a senha é aleatória e fica num arquivo legível só pelo root, e a frase de recuperação é mostrada ao final;
5. gera `/etc/thecoin/thecoind.toml` e um serviço systemd endurecido (`ProtectSystem=strict`, `NoNewPrivileges`, sem capabilities), abre a porta P2P no ufw quando ativo;
6. instala o comando `thecoin` e inicia o nó, que começa a sincronizar e minerar.

Rodar de novo atualiza os binários mantendo configuração e carteira.

Opção	Variável	Efeito
<code>--network mainnet testnet</code>	<code>THECOIN_NETWORK</code>	Rede (padrão mainnet).
<code>--miner-address <tc1...></code>	<code>THECOIN_MINER_ADDRESS</code>	Recompensas para um endereço existente, sem criar carteira.
<code>--no-mine</code>	<code>THECOIN_NO_MINE=1</code>	Nó completo sem minerar.
<code>--threads <n></code>	<code>THECOIN_THREADS</code>	Threads de mineração (0 = núcleos - 1).
<code>--version <x.y.z latest></code>	<code>THECOIN_VERSION</code>	Versão a instalar.
<code>--from-source</code>	<code>THECOIN_FROM_SOURCE=1</code>	Compilar em vez de baixar.
<code>--public-api</code>	<code>THECOIN_PUBLIC_API=1</code>	API em 0.0.0.0 (padrão: só 127.0.0.1).
<code>--yes</code> , <code>-y</code>	<code>THECOIN_YES=1</code>	Não interativo: nunca pergunta, aceita padrões.

Tabela 17: Opções do instalador (`installer/install.sh`).

16.3 O comando `thecoin`

Comando	Função
<code>thecoin status</code>	Estado do nó, sincronização, pares, mineração e saldo.
<code>thecoin logs</code>	Acompanha os logs do serviço.
<code>thecoin address / balance</code>	Endereço de mineração e saldo da carteira.
<code>thecoin mnemonic</code>	Mostra a frase de recuperação (guarde em segredo).
<code>thecoin restart</code>	Reinicia o serviço.
<code>thecoin signals / signal <id> / unsignal <id></code>	Sinalização de governança (Seção 14).
<code>thecoin update</code>	Atualiza para a última versão mantendo configuração e carteira.
<code>thecoin uninstall [--purge]</code>	Remove o nó (<code>--purge</code> apaga dados da cadeia e configuração; arquivos de carteira nunca são apagados).
<code>thecoin version</code>	Versões instaladas.

Tabela 18: Comando auxiliar para operar um nó instalado. Operações de carteira: `thecoin-wallet --help`.

16.4 Requisitos

Recurso	Mínimo	Recomendado
CPU	1 vCPU x86_64 ou ARM64	2+ vCPU
Memória	1 GB (com swap)	2 GB
Disco	5 GB (nó podado)	20 GB SSD (arquivo: ver Seção 20)
Rede	porta TCP 7333 aberta	IP público estável
Sistema	Linux com systemd	Ubuntu/Debian LTS

Tabela 19: Requisitos de um nó minerador na fase de prova de trabalho.

17 Carteira e padrões para desenvolvedores

A carteira de referência usa exclusivamente padrões abertos, para que qualquer carteira compatível derive os mesmos endereços a partir das mesmas palavras:

- **BIP-39**: frase de recuperação de 12 ou 24 palavras em inglês.
- **SLIP-0010 Ed25519**: caminho `m/44'/7333'/conta'/0'/índice'` (todos endurecidos). As chaves de anel de privacidade também derivam da frase.
- **Arquivo de carteira v1**: JSON com a frase cifrada por ChaCha20-Poly1305 com chave derivada da senha por Argon2id (64 MiB, 3 passes), gravado com permissão 0600.
- **Transações**: codificação Borsh documentada byte a byte, com vetores de teste; taxa calculada com as mesmas funções de consenso (`required_fee`).
- **Comandos**: `send`, `batch`, `pay` (URI), `bump-fee`, `confirmations`, `alerts`, `fees`, `contract ...` (nativos e TCCL), `privacy ...`, `gov ...`, com as opções globais `--priority`, `--replaceable` e `--dry-run`.

A biblioteca `thecoin-core` não tem I/O e compila para qualquer plataforma suportada pelo Rust, inclusive WebAssembly, o que permite carteiras web e móveis reutilizarem exatamente as regras de consenso. A documentação do repositório (`docs/WALLET_DEVELOPERS.md`, `docs/PROTOCOL.md`, `docs/API.md`, `docs/CONTRACTS.md`) descreve tudo o que é necessário para criar novas carteiras.

18 API e integração com o site

Cada nó expõe uma API REST JSON (`/api/v1`) com status da rede (incluindo o multiplicador de congestionamento), oferta, taxas e níveis de prioridade, blocos, transações com recibos e eventos, endereços e histórico, contratos nativos, contratos TCCL (`/program/{addr}`) e consultas `view` gratuitas), simulação de transações, estimador de confirmações, alertas de gasto duplo, propostas de governança e parâmetros, mempool, pares e mineração. Valores são inteiros em moedas, hashes em hexadecimal e endereços em bech32m.

A API vem ligada apenas em `127.0.0.1`. Para o site the-coin.cloud, hospedado em outra máquina, os nós seed expõem a API somente ao IP do servidor web, e o nginx do site faz **proxy** com TLS, cache curto, limite de requisições e **failover** entre os dois nós. O explorador, o painel de governança e as estatísticas da página inicial consomem essa API — não há banco de dados central: os dados exibidos são os mesmos que qualquer pessoa pode verificar rodando o próprio nó.

19 Segurança

Ameaça	Mitigação
Alterar transações confirmadas	Cada bloco compromete transações (Merkle), estado (LtHash) e o bloco anterior; reescrever o histórico exige refazer todo o trabalho acumulado.
Gasto duplo / ataque de 51%	Cadeia de maior trabalho; reorganizações acima de 720 blocos recusadas; checkpoints; recompensa liberada 25% após 100 blocos e 75% após 1000; estimador de confirmações por valor; alertas de gasto duplo; substituição só com opt-in.
Inundação de transações	Taxa base por transação e por byte; multiplicador de congestionamento até 1000× com sobretaxa queimada; mempool limitado com expulsão por prioridade.
Contratos abusivos	Combustível pago e limitado por transação e por bloco; limites de memória, profundidade e tamanho; depósito de armazenamento; sem chamadas entre contratos (sem reentrância); falha reverte tudo menos a taxa.
Inflação indevida	Emissão calculada deterministicamente e verificada por todos os nós; teto de 50M checado em cada bloco; aritmética com verificação de overflow (também na VM).
Replay de transações	Nonce por conta e <code>chain_id</code> por rede.
Maleabilidade	Codificação canônica e Ed25519 estrito.
Manipulação de timestamp	Median time past, deriva máxima de 180 s, limites do LWMA e variação máxima de 2× por bloco.
Negação de serviço	Validação barata antes da cara e PoW verificada antes de aceitar blocos; pool de órfãos limitado; forks mais antigos que a profundidade máxima descartados; filas de envio limitadas; pontuação e banimento de pares; limites da API; alertas falsos de gasto duplo impossíveis (exigem duas assinaturas válidas).
Eclipse / Sybil na rede	Conexões de saída escolhidas pelo próprio nó, múltiplas seeds, limite por IP, gerenciador de endereços persistente; alturas anunciadas por pares tratadas como alegações.
Captura da governança	Duas câmaras independentes, moedas de voto bloqueadas, quórum, depósitos anti-spam, limites rígidos dos parâmetros, supply e emissão imutáveis.

Ameaça	Mitigação
Roubo de chaves	Nós não precisamos de chaves de usuários; o instalador guarda apenas a carteira de recompensas, cifrada com Argon2id + ChaCha20-Poly1305 (no modo não interativo, a senha fica num arquivo legível só pelo root); serviço isolado por systemd.
Corrupção de dados	Transações ACID, compromisso de estado verificado a cada bloco e na inicialização.

Tabela 20: Modelo de ameaças e mitigações.

Uma rede de prova de trabalho pequena é mais vulnerável a ataques de maioria do que redes estabelecidas. A resposta de curto prazo são as salvaguardas acima; a de longo prazo é crescimento — cada VPS minerando aumenta o custo de um ataque — e, se aprovada, a finalidade por stake da fase híbrida (Seção 21).

20 Desempenho e escalabilidade

As medições de CPU abaixo foram feitas com a implementação de referência (compilação `release`) num VPS OVH de 2 vCPU Haswell, 3,7 GB de RAM e disco SSD de rede — exatamente o tipo de máquina para a qual a rede foi projetada. As medições de processamento vêm da v0.1 (o caminho de validação de transferências não mudou de forma relevante); as de disco foram refeitas na v0.2, com recibos e o novo índice de endereços.

Medição	Resultado	Versão
Transferência simples (sem memo)	159 bytes	0.2
Cabeçalho de bloco	180 bytes	0.1
CoinHash (16 MiB) por tentativa	12,0 ms · 83–87 H/s por thread	0.1
Verificação sem estado + Ed25519 estrito	≈ 14 400 transações/s por núcleo	0.1
Aplicar bloco de 790 kB com 5 000 transferências (inclui assinaturas)	0,39 s (≈ 12 900 tx/s)	0.1
Atualizar o LtHash para 5 002 registros alterados	62 ms	0.1
Processar e gravar bloco vazio (validação + banco)	0,39 ms	0.1
Memória residente do nó minerando (1 thread)	≈ 42 MB	0.1
Disco por bloco vazio em regime (cabeçalho, índices)	≈ 765 bytes (≈ 400 MB/ano)	0.1
Disco por transação, nó arquivo com índice de endereços — dados	≈ 430 bytes	0.2
espaço alocado pelo redb (antes de <code>thecoind compact</code>)	≈ 792 bytes	0.2
Disco por transação, nó arquivo sem índice de endereços — dados	≈ 348 bytes	0.2
espaço alocado pelo redb (antes de <code>thecoind compact</code>)	≈ 633 bytes	0.2

Tabela 21: Desempenho medido da implementação de referência. Disco: 100 000 transferências para contas novas, em regime (dados de undo já descartados), teste `crates/node/tests/storage_growth.rs`.

20.1 Capacidade de processamento

Com o limite padrão de 1 MB por bloco (`max_block_bytes`) e blocos de 60 s, cabem $\lfloor (1\,000\,000 - 184)/159 \rfloor = 6\,288$ transferências por bloco — cerca de **105 transações por segundo** sustentadas na camada base, ou ≈ 9,05 milhões por dia. Um bloco cheio é validado em menos de meio segundo por um único núcleo modesto, menos de 1% do intervalo entre blocos: mesmo com a rede no limite, a validação não compete com a

mineração nem torna o nó lento. O limite de bytes é um parâmetro de governança (250 kB a 8 MB); contratos têm ainda o limite de combustível por bloco (`max_block_fuel`, 50 milhões por padrão, entre 5 e 500 milhões).

20.2 Armazenamento: o que ocupa espaço

- **Estado:** proporcional às contas ativas (≈ 60 bytes por conta) e ao armazenamento dos contratos — este último sempre coberto por depósito reembolsável (Seção 12). Não cresce com o histórico.
- **Cabeçalhos:** ≈ 765 bytes por bloco com índices, ≈ 400 MB por ano, sempre mantidos, pois provam a cadeia de trabalho.
- **Corpos de blocos:** só existem quando há transações e são comprimidos com `zstd`; nós podados mantêm apenas os mais recentes (padrão `prune_keep` = 10 000 blocos, mínimo 1 000).
- **Recibos:** um por transação confirmada (taxa, parte queimada, endereços afetados, eventos), comprimidos com `zstd`. A poda remove os recibos e as entradas do índice de transações junto com o corpo do bloco.
- **Índice de endereços:** opcional (`storage.address_index`); na v0.2 cada entrada é só a chave (endereço + altura + posição, 32 bytes) e o valor é vazio — o `txid` é lido do próprio bloco. Nós podados desativam o índice automaticamente.
- **Dados de undo:** mantidos apenas para os últimos 736 blocos (720 + margem de 16).

Dados × espaço alocado. O `redb` é um banco copy-on-write que pré-aloca páginas para crescer sem fragmentar. Por isso o arquivo `chain.redb` ocupa mais do que os dados que contém (≈ 792 contra ≈ 430 bytes por transação com índice). O excedente é recuperável a qualquer momento com o nó parado: `thecoind compact`. As projeções abaixo usam os dois valores: **compactado** (dados) e **sem compactar** (pior caso).

20.3 Projeções de disco

Seja D o disco disponível, b os bytes por transação e $H \approx 525\,960 \times 765 \text{ B} \approx 0,40 \text{ GB/ano}$ o custo fixo dos cabeçalhos (valor medido na v0.1, conservador). Então:

$$N_{\text{tx}} = \frac{D}{b} \quad \text{anos}(T) = \frac{D}{365,25 \cdot T \cdot b + H}$$

onde T é o número de transações por dia. Estado e dados de undo (algumas centenas de MB para milhões de contas) não estão incluídos.

Configuração	bytes/tx	20 GB	50 GB	100 GB
Arquivo + índice, compactado	430	46,5 mi	116,3 mi	232,6 mi
Arquivo + índice, sem compactar	792	25,3 mi	63,1 mi	126,3 mi
Arquivo sem índice, compactado	348	57,5 mi	143,7 mi	287,4 mi
Arquivo sem índice, sem compactar	633	31,6 mi	79,0 mi	158,0 mi

Tabela 22: Quantas transações cabem no disco ($N_{\text{tx}} = D/b$, em milhões), ignorando cabeçalhos.

Transações/dia	Configuração	20 GB	50 GB	100 GB
1 000	Arquivo + índice, compactado	35,8 anos	89,4 anos	178,8 anos
	Arquivo + índice, sem compactar	28,9 anos	72,3 anos	144,6 anos
	Arquivo sem índice, compactado	37,8 anos	94,4 anos	188,9 anos
	Arquivo sem índice, sem compactar	31,6 anos	78,9 anos	157,8 anos
10 000	Arquivo + índice, compactado	10,1 anos	25,3 anos	50,7 anos
	Arquivo + índice, sem compactar	6,1 anos	15,2 anos	30,3 anos
	Arquivo sem índice, compactado	12,0 anos	29,9 anos	59,8 anos
	Arquivo sem índice, sem compactar	7,4 anos	18,4 anos	36,8 anos

Transações/dia	Configuração	20 GB	50 GB	100 GB
100 000	Arquivo + índice, compactado	1,2 anos	3,1 anos	6,2 anos
	Arquivo + índice, sem compactar	249 dias	1,7 anos	3,4 anos
	Arquivo sem índice, compactado	1,5 anos	3,8 anos	7,6 anos
	Arquivo sem índice, sem compactar	311 dias	2,1 anos	4,3 anos
1 000 000	Arquivo + índice, compactado	46 dias	116 dias	232 dias
	Arquivo + índice, sem compactar	25 dias	63 dias	126 dias
	Arquivo sem índice, compactado	57 dias	143 dias	286 dias
	Arquivo sem índice, sem compactar	32 dias	79 dias	158 dias
Blocos sempre cheios (9054720 tx/dia), arquivo + índice compactado		5 dias	13 dias	26 dias

Tabela 23: Anos até encher o disco de um nó arquivo, $\text{anos}(T) = D/(365,25 Tb + H)$.

A leitura prática: um nó arquivo com 50 GB guarda cerca de 116 milhões de transferências (compactado) e acompanha uma rede de 10 000 transações por dia por mais de 25 anos. Mesmo a 100 000 transações por dia — ordem de grandeza de uma rede nacional de pagamentos pequena — 100 GB duram mais de 6 anos. A capacidade máxima da camada base (blocos sempre cheios) consumiria $\approx 3,9$ GB por dia num nó arquivo com índice; esse cenário é tratado pelo multiplicador de congestionamento (taxas sobem exponencialmente) e pela poda.

Nó podado. Um minerador comum não precisa do histórico: com poda, o disco fica limitado a cabeçalhos ($\approx 0,4$ GB/ano) + estado + os últimos `prune_keep` corpos de bloco. No pior caso (blocos de 1 MB sempre cheios) são ≈ 10 GB para 10 000 blocos, ou ≈ 1 GB com o mínimo de 1 000; com uso real, muito menos. Um disco de 5–10 GB atende um nó podado por muitos anos; os nós arquivo (seeds e os que alimentam o site) guardam o histórico completo, que permanece público e verificável.

20.4 Mempool persistente e sincronização

Ao desligar, o nó grava as transações pendentes em `mempool.dat` (formato versionado, escrita atômica via arquivo temporário) e, ao iniciar, revalida cada uma contra o estado atual antes de readmiti-la: uma reinicialização ou atualização não faz transações de usuários desaparecerem.

Um novo nó baixa blocos em lotes de 500, verifica a prova de trabalho em paralelo em todos os núcleos e aplica cada bloco em ordem, conferindo a raiz de estado. O custo dominante é a prova de trabalho (12 ms por bloco por núcleo): um ano de cadeia exige $\approx 1,75$ hora de CPU, ou ≈ 53 minutos numa máquina de 2 núcleos. A sincronização por **snapshot** de estado, prevista originalmente para a 0.2, foi adiada (Seção 22).

21 Roadmap de consenso: PoW → Híbrido → PoS

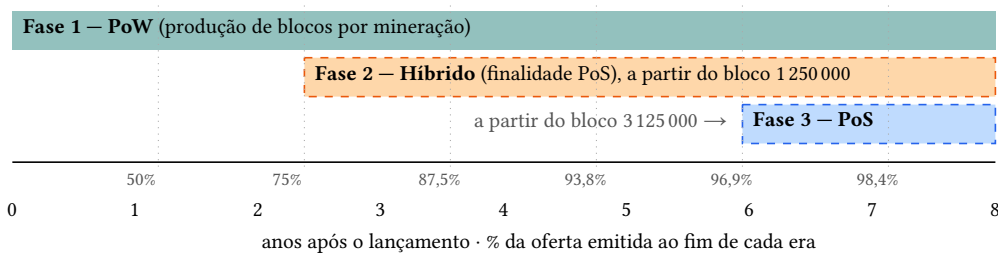
Plano, não implementação. Esta seção apresenta o plano aprovado para discussão pública em `docs/ROADMAP.md`. **Nada dele está implementado na v0.2.** Cada transição é uma atualização de consenso e só acontece com aprovação da governança bicameral (detentores e mineradores) e com as condições objetivas listadas abaixo.

A The Coin nasce em prova de trabalho e evolui, em três fases, para prova de participação com finalidade rápida. A ordem não é por moda: cada mecanismo é usado quando é o mais seguro para o momento da rede.

	Fase 1 — PoW	Fase 2 — Híbrido	Fase 3 — PoS
Quem produz blocos	mineradores (CoinHash)	mineradores (CoinHash)	validadores com stake, em rodízio

	Fase 1 — PoW	Fase 2 — Híbrido	Fase 3 — PoS
Quem finaliza	ninguém (probabilístico, reorg máx. 720)	validadores votam checkpoints a cada 120 blocos	validadores (BFT) a cada bloco
Tempo até irreversível	de 1 a 720 blocos (≈ 12 h), conforme o valor	≤ 2 épocas (≈ 4 h), depois nunca reverte	≈ 2 blocos
Tempo de bloco	60 s	60 s	10 s (definido na atualização da fase 3)
Recompensa	100% mineradores	dividida (começa 90/10, no máximo 50/50)	100% validadores
Mais cedo possível	bloco 0	bloco 1 250 000 ($\approx 2,4$ anos)	bloco 3 125 000 ($\approx 5,9$ anos)

Tabela 24: As três fases do plano de consenso (docs/ROADMAP.md).

Figura 6: Alturas **mínimas** das fases (tracejado = só com aprovação e condições cumpridas).

21.1 Por que esta ordem

- PoW primeiro — distribuição justa.** No começo ninguém tem moedas. Em PoS puro, quem começa com moedas controlaria a rede para sempre. PoW com CoinHash distribui a emissão a quem contribui com CPU, inclusive VPS pequenas, sem pré-mineração.
- Híbrido depois — segurança extra sem trocar o motor.** Quando $\approx 75\%$ da emissão já foi distribuída (fim da era 2), existe base ampla de detentores. Os validadores passam a **finalizar** checkpoints: um ataque de 51% de **hashrate** não consegue mais reverter blocos finalizados. O PoW continua produzindo blocos, então um erro no PoS não para a rede.
- PoS no final — rapidez e eficiência.** Com $\approx 97\%$ da emissão distribuída (fim da era 5), a recompensa de bloco já é pequena e a segurança paga pela emissão cairia. A partir daí é mais seguro e barato proteger a rede com stake que pode ser punido, com confirmação em segundos — mantendo hardware modesto.

21.2 Fase 1 — PoW (atual)

Já implementado na v0.2: CoinHash e LWMA-1 com aquecimento e limite de $2\times$ (Seção 5.2); recompensa com cooldown de 25% / 75% (Seção 6.3); reorganização máxima de 720 blocos, suporte a checkpoints (lista preenchida a partir do lançamento), alertas de gasto duplo e estimador de confirmações (Seção 8); governança com sinalização de mineradores e propostas de atualização de software — o mecanismo que ativará as próximas fases.

21.3 Fase 2 — Híbrido: PoW + finalidade PoS

Ativação: proposta de atualização de software aprovada pela governança e todas as condições: altura $\geq 1\,250\,000$; implementação com ≥ 6 meses de testnet sem falha de segurança e auditoria externa; stake registrado $\geq 10\%$ da oferta circulante, em ≥ 50 validadores.

- **Stake:** nova ação `Stake { amount, validator_key }`; mínimo de 1 000 TCN (parâmetro de governança). Pequenos detentores **delegam** a um validador sem entregar as chaves. Saída com período de desbloqueio de 43 200 blocos (30 dias).
- **Épocas** de 120 blocos (≈ 2 h). Ao fim de cada época os validadores assinam (Ed25519) um voto para o checkpoint da época. Quando votos que somam $\geq 2/3$ do stake justificam dois checkpoints seguidos, o primeiro fica **finalizado** — o modelo **Casper FFG** [17].
- **Regra de escolha:** a cadeia deve conter o último checkpoint finalizado; entre as que o contêm, vence a de maior trabalho acumulado. Blocos finalizados nunca são revertidos.
- **Recompensa:** começa em 90% mineradores / 10% validadores e move 10 pontos percentuais por era, até no máximo 50/50 (se a fase 3 começar na altura mínima, a divisão chega a 70/30). A emissão total e o teto de 50 000 000 TCN não mudam.
- **Punição (slashing):** votar em dois checkpoints conflitantes na mesma época, ou votos que se “cercam”, queima 10% do stake e remove o validador. Validadores inativos perdem recompensa, sem queima — a rede nunca para, porque o PoW continua.
- **Anti-concentração:** o peso de voto de um validador é limitado a 5% do total; stake acima disso não aumenta poder nem recompensa.
- **Ataques de longo alcance:** nós novos sincronizam a partir de um checkpoint finalizado recente, distribuído no software e no site (**weak subjectivity**).

21.4 Fase 3 — PoS com finalidade rápida

Ativação: nova proposta aprovada e todas as condições: altura $\geq 3\,125\,000$; ≥ 2 anos de Fase 2 sem nenhuma falha de finalidade; stake $\geq 33\%$ da oferta circulante, em ≥ 100 validadores independentes; coeficiente de Nakamoto ≥ 7 (são necessários ≥ 7 validadores para somar $1/3$ do stake).

- **Produção:** líderes escolhidos por stake com **VRF** [20] (sorteio verificável e imprevisível), rodízio a cada slot de 10 s. O tempo de bloco é uma constante de consenso, não um parâmetro de votação comum: a mesma atualização multiplica por 6 todos os intervalos contados em blocos (halving, maturação, desbloqueio de recompensas, votação), para que o cronograma de emissão em tempo e o teto não mudem.
- **Finalidade:** BFT no estilo HotStuff/Tendermint [18, 19] — um bloco com votos de $\geq 2/3$ do stake é final em ≈ 2 blocos. Até $1/3$ de validadores maliciosos não reverte nada.
- **PoW removido:** o último bloco PoW vira o checkpoint de gênese da fase. Mineradores migram fazendo stake das recompensas acumuladas; o instalador de uma linha passa a configurar um validador de stake em vez de um minerador.
- **Recompensa:** a emissão restante e as gorjetas vão para validadores e delegadores; a queima da sobretaxa de congestionamento continua.
- **Hardware:** continua modesto (mínimo 1 vCPU / 1 GB; recomendado 2 vCPU / 4 GB / SSD). Um bloco totalmente cheio executa em $\approx 1,2$ s numa VPS de 2 vCPU; para blocos de 10 s os limites por bloco são reduzidos na mesma proporção, e a governança só aumenta limites com medições publicadas.
- **Contratos TCCL** continuam iguais: VM, taxas e depósitos não dependem do mecanismo de consenso.

21.5 O que não muda em nenhuma fase

- O teto de 50 000 000 TCN e o cronograma de emissão.
- Todo o histórico desde o bloco 0 continua público e verificável.
- Endereços, carteiras, contratos e a linguagem TCCL.
- Nenhuma fase é ativada sem votação dos detentores e sinalização da rede.

21.6 Riscos e tratamento

Risco	Tratamento
Bug no novo consenso	A fase híbrida mantém o PoW produzindo blocos; testnet e auditoria antes da ativação.

Risco	Tratamento
Stake concentrado em corretoras	Limite de 5% de peso por validador, delegação livre, coeficiente de Nakamoto como condição.
“Nada em jogo” / votos duplos	Slashing de 10% com prova enviada por qualquer nó.
Ataque de longo alcance	Checkpoints finalizados e sincronização a partir de checkpoint recente.
Mineradores contra a transição	A ativação exige também sinalização dos mineradores.

Tabela 25: Riscos da transição de consenso.

22 Lançamento da rede e roteiro de software

22.1 Plano de lançamento

1. Publicar código, binários com checksum, instalador e este whitepaper.
2. Operar uma **testnet** pública para validação por desenvolvedores.
3. Colocar no ar os dois nós seed (`seed1` e `seed2.the-coin.cloud`) e o site em máquina separada.
4. Fixar o timestamp da gênese da mainnet (13 de setembro de 2026, 00:00 UTC) e iniciar a mineração nos nós seed.
5. Abrir o instalador ao público; cada novo nó sincroniza e passa a minerar automaticamente.
6. Após as primeiras semanas, adicionar checkpoints de alturas consolidadas em novas versões.

22.2 Roteiro

- **Entregue na v0.2** — novo modelo de taxas e prioridade, confirmação segura, TCCL (que substitui a VM WebAssembly prevista para a v0.3), privacidade por contratos, armazenamento esquema 2, instalação em uma linha.
- **Adiado da v0.2** — criptografia do transporte P2P (Noise), sincronização rápida por **snapshot** de estado verificado pelo `state_root`, carteira gráfica e web, provas Merkle para clientes leves.
- **Próximas versões** — os itens adiados acima; canais de pagamento sobre HTLC, integração com exchanges e ferramentas para comércio (do roteiro da v0.1).
- **Consenso** — Fases 2 e 3 conforme a Seção 21, somente com aprovação da governança.
- **Contínuo** — auditorias externas, programa de recompensa por falhas, diversificação de implementações.

23 Conclusão

The Coin combina a robustez comprovada da prova de trabalho e de uma política monetária fixa com escolhas técnicas voltadas a máquinas modestas: uma prova de trabalho memory-hard que valoriza CPUs comuns, dificuldade que reage em minutos, validação barata com compromisso de estado homomórfico, armazenamento compacto e um nó que não disputa recursos com o próprio sistema. A versão 0.2 torna a rede útil em escala: taxas de cerca de 0,00003 TCN por transferência em uso normal, que crescem exponencialmente sob ataque, confirmação rápida com proteções explícitas contra gasto duplo, contratos inteligentes com regras de segurança impostas pelo compilador, privacidade opcional com limites claramente declarados e instalação de um validador com um único comando. O caminho para a finalidade por stake está traçado, com condições objetivas e sempre sujeito à decisão conjunta de detentores e mineradores. O código é aberto, os padrões são documentados e qualquer pessoa pode rodar um nó, publicar um contrato ou propor melhorias.

Apêndice A — Parâmetros das redes

Parâmetro	Mainnet	Testnet	Regtest
Magic P2P	TCN1	TCNT	TCNR
chain_id	0x54430001	0x54430002	0x54430003
Portas P2P / API	7333 / 7334	17333 / 17334	27333 / 27334
Prefixo de endereço	tc	tct	tcr
CoinHash	16 MiB, t=1	16 MiB, t=1	64 KiB, t=1
Alvo da gênese / limite	$2^{256}/2^{13} / 2^{256}/2^8$	$2^{256}/2^{10} / 2^{256}/2^6$	máximo, sem ajuste
Tempo de bloco / janela LWMA (mín.)	60 s / 60 (6)	60 s / 60 (6)	60 s / —
Recompensa inicial	40 TCN	40 TCN	40 TCN
Halving	625 000 blocos	625 000 blocos	150 blocos
Liberação da recompensa 25% / 75%	100 / 1 000	100 / 1 000	5 / 12
Reorganização máxima	720	720	720
Bloco máximo / combustível (padrão)	1 MB / 50 M	1 MB / 50 M	1 MB / 50 M
base_fee / fee_per_kb / fee_per_kfuel	1 000 / 10 000 / 1 000	1 000 / 10 000 / 1 000	100 / 1 000 / 100
storage_deposit_per_kb	100 000	100 000	10 000
Congestionamento	1× – 1 000×	1× – 1 000×	1× – 1 000×
Depósito de proposta	100 TCN	10 TCN	1 TCN
Votação / ativação	20 160 / 2 880	1 440 / 720	20 / 5
Quórum / aprovação / mineradores	10% / 66,67% / 60%	5% / 66,67% / 50%	10% / 66,67% / 50%

Apêndice B — Referências

1. S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
2. A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson. *RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*, 2021.
3. J. O'Connor, J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn. *BLAKE3: one function, fast everywhere*, 2020.
4. S. Josefsson, I. Liusvaara. *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)*, 2017.
5. M. Bellare, D. Micciancio. *A New Paradigm for Collision-free Hashing: Incrementality at Reduced Cost*, EUROCRYPT 1997.
6. K. Lewi, W. Kim, I. Maykov, S. Weis. *Securing Update Propagation with Homomorphic Hashing (LtHash)*, 2019.
7. Zawy12. *LWMA difficulty algorithm*, 2018–2019.
8. P. Wuille. *BIP-350: Bech32m format for v1+ witness addresses*, 2020.
9. M. Palatinus et al. *BIP-39: Mnemonic code for generating deterministic keys*, 2013.
10. SatoshiLabs. *SLIP-0010: Universal private key derivation from master private key*, 2016.
11. NEAR Protocol. *Borsh: Binary Object Representation Serializer for Hashing*.
12. V. Buterin, E. Conner, R. Dudley, M. Slipper, I. Norden, A. Bakhta. *EIP-1559: Fee market change for ETH 1.0 chain*, 2019.
13. D. A. Harding, P. Todd. *BIP-125: Opt-in Full Replace-by-Fee Signaling*, 2015.
14. M. Corallo. *BIP-152: Compact Block Relay*, 2016.
15. A. Back. *Ring signature efficiency (bLSAG)*, 2015; S. Noether, A. Mackenzie. *Ring Confidential Transactions*, Ledger 2016.
16. H. de Valence, J. Grigg, G. Tankersley, F. Valsorda, I. Lovercruft, M. Hamburg. *The ristretto255 and decaf448 Groups*, RFC 9496, 2023.
17. V. Buterin, V. Griffith. *Casper the Friendly Finality Gadget*, 2017.
18. M. Yin, D. Malkhi, M. K. Reiter, G. Golan Gueta, I. Abraham. *HotStuff: BFT Consensus with Linearity and Responsiveness*, PODC 2019.
19. E. Buchman, J. Kwon, Z. Milosevic. *The latest gossip on BFT consensus (Tendermint)*, 2018.
20. S. Micali, M. Rabin, S. Vadhan. *Verifiable Random Functions*, FOCS 1999.

-
21. The Coin. `docs/ROADMAP.md`, `docs/PROTOCOL.md`, `docs/CONTRACTS.md`, `docs/GOVERNANCE.md` e *TCCL Cookbook* (`docs/tccl/tccl-cookbook.pdf`), repositório `LucasBolla94/thecoin`.